



HARBORVIEW GRAND

BOUTIQUE WATERFRONT RESORT & SPA

Full-Stack AI Hotel Guest Assistant Complete Project Report & Documentation

Technical Design · Architecture · API Reference · Test Evidence

DEVELOPER Sumit Kumar Singh

REPOSITORY github.com/Sumitboii/HARBORVIEW-GRAND

LIVE DEMO harborview-grand.onrender.com

ASSIGNMENT Full-Stack AI-Powered Hotel Guest Assistant

CONTENTS

1	Assignment Overview	3
2	What Was Built	3
3	Tech Stack — What, Why & How	4
4	Project Structure	5
5	Architecture & Data Flow	6
6	Backend Deep Dive	7
7	Frontend Deep Dive	8
8	AI Integration	9
9	Knowledge Base	10
10	API Reference	11
11	Test Coverage	12
12	Assignment Completion Checklist	13
13	Design Decisions & Trade-offs	14
14	How to Run Locally	15

1. ASSIGNMENT OVERVIEW

The assignment required building a full-stack AI-powered **Hotel Guest Assistant** — a chat interface where hotel guests can ask natural-language questions about hotel policies, amenities, rooms, and services, and check live room availability.

REQUIRED QUESTIONS (FROM THE ASSIGNMENT SPEC)

#	Question	Type	Status
1	What time is check-in?	FAQ / Policy	PASS
2	Does the hotel have a swimming pool?	Amenity	PASS
3	Which room is suitable for three guests?	Room Info	PASS
4	Is breakfast included?	Policy	PASS
5	What is the cancellation policy?	Policy	PASS
6	Do you have rooms available for a given date?	Availability	PASS

All six questions are implemented, tested, and accessible via clickable suggestion chips on the chat interface.

2. WHAT WAS BUILT

Harborview Grand is a two-panel luxury hotel concierge system. The left sidebar displays hotel information (schedule, contact, status) and the right panel is the full chat interface powered by Google Gemini.

KEY FEATURES DELIVERED

- **Natural language Q&A** — guests type any question; the system retrieves relevant hotel facts and generates a natural response
- **Room availability checker** — structured form for check-in/check-out dates and guest count; returns per-room inventory with pricing
- **Multi-turn conversation** — conversation ID maintained across messages; follow-ups use prior context

- **Hallucination prevention** — LLM grounded strictly in retrieved facts; availability numbers never pass through the model
- **Graceful fallbacks** — works without any API key; handles API rate limits and errors silently
- **Intent detection** — keyword/pattern routing before any LLM call
- **Suggestion chips** — all six required questions shown as clickable chips on load
- **Deployed on Render** — backend and frontend as separate services via [render.yaml](#) Blueprint

3. TECH STACK — WHAT, WHY & HOW

BACKEND

Technology	What it is	Why chosen	How used
Node.js	JavaScript runtime	Matches frontend language; fast async I/O	Runs the Express server
Express.js	Web framework	Minimal and fast; single endpoint doesn't need a heavy framework	Handles POST /api/chat, GET /api/health, error handlers
dotenv	Env loader	Standard way to manage secrets	Loads GEMINI_API_KEY, PORT from .env
cors	CORS middleware	Required for browser→API cross-origin requests	Configured with origin allowlist
Google Gemini API	LLM (AI model)	Free tier; strong instruction-following; fast latency	Natural-language answers; KB fallback if unavailable
Jest + Supertest	Test runner + HTTP client	Industry standard for Node.js	27 tests across 3 suites

FRONTEND

Technology	What it is	Why chosen	How used
Next.js 14	React framework	Preferred by assignment brief; SSR + fast HMR	App entry point, page routing, production builds
React 18	UI library	Industry standard component model	ChatWindow, MessageBubble, AvailabilityForm
CSS (globals.css)	Styling	No extra dependencies; full layout control	Two-column hotel layout, animations, responsive design
Playwright	E2E testing	Official Next.js E2E tool	4 end-to-end test scenarios

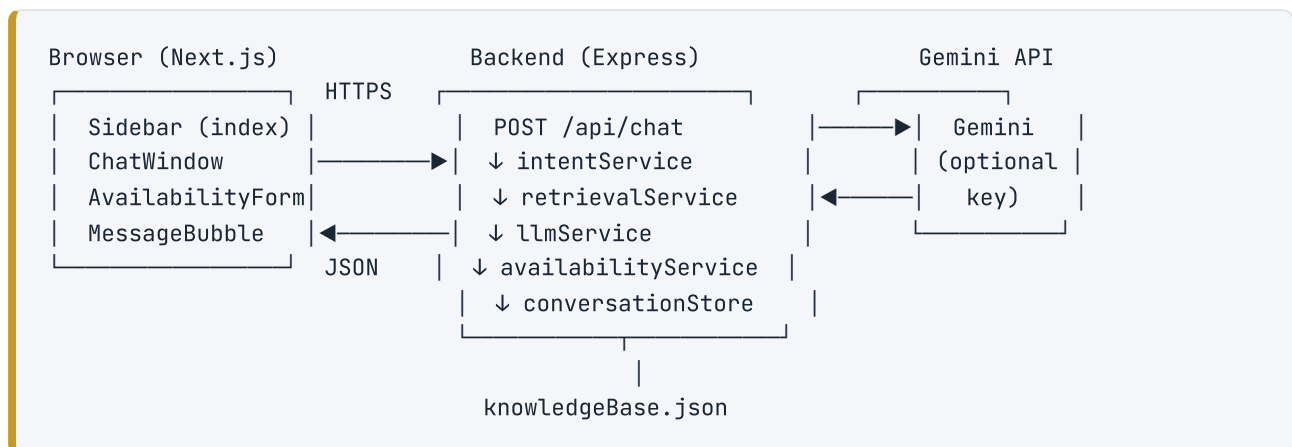
INFRASTRUCTURE

Technology	Why chosen	How used
GitHub	Version control; required for Render	Source of truth; auto-deploy triggers
Render	Free Node.js hosting; Blueprint YAML deploy	Two services: API + frontend static build
render.yaml	Declarative infrastructure-as-code	Defines both services, env vars, build/start commands

4. PROJECT STRUCTURE

```
hotel-assistant/
├─ backend/
│  └─ src/
│     │ app.js           Express factory, CORS, error handlers
│     │ server.js        Entry point (binds PORT)
│     │ routes/
│     │ │ chat.js        POST /api/chat – intent routing hub
│     │ │ services/
│     │ │ │ intentService.js  Keyword/regex intent classifier
│     │ │ │ retrievalService.js TF-IDF keyword retrieval from KB
│     │ │ │ llmService.js     Gemini API calls + KB fallback
│     │ │ │ availabilityService.js Deterministic room inventory check
│     │ │ │ conversationStore.js In-memory conversation history (10 turns)
│     │ │ data/
│     │ │ │ knowledgeBase.json Hotel facts: rooms, policies, amenities, FAQs
│     └─ _tests_/
│        │ chat.test.js      12 API integration tests
│        │ availabilityService.test.js 12 unit tests
│        └─ retrieval_intent.test.js 3 intent / retrieval unit tests
├─ frontend/
│  │ pages/
│  │ │ index.js           Two-column layout + sidebar
│  │ │ _app.js            Global CSS import
│  │ │ components/
│  │ │ │ ChatWindow.js    Chat state, message sending
│  │ │ │ MessageBubble.js Message renderer (text/availability/typing)
│  │ │ │ AvailabilityForm.js Date picker + results display
│  │ │ styles/globals.css All styles – layout, chat, animations
│  │ └─ tests/e2e.spec.js 4 Playwright end-to-end tests
├─ docs/
│  │ PROJECT_REPORT.md / .pdf This document
│  │ ARCHITECTURE.md         API contract, data flow, AI design
│  │ REQUIREMENTS.md        Requirement matrix with IDs and evidence
│  │ TEST_PLAN.md           16 test scenarios with observed results
│  └─ REQUIREMENT_AUDIT.md   Final PASS / out-of-scope audit
├─ render.yaml
└─ README.md                Quick-start, API examples, product notes
```

5. ARCHITECTURE & DATA FLOW



REQUEST LIFECYCLE

1. Guest submits message via chat input or clicks a suggestion chip
2. Frontend sends `POST /api/chat` with `{ message, conversationId? }`
3. Backend validates — empty message returns `400 INVALID_REQUEST`
4. **Intent detection** — `intentService.detectIntent()` classifies the message into: *greeting*, *gratitude*, *booking_request*, *availability*, *availability_needs_details*, *general_question*
5. **Retrieval** — `retrievalService.retrieve()` scores every KB chunk by keyword overlap; top-N chunks selected
6. **LLM call** — `llmService.callLLM()` sends question + context to Gemini; if unavailable → falls back to `mockAnswer()`
7. Turn appended to in-memory store keyed by `conversationId`
8. Structured JSON response returned → rendered as bubble or availability card

6. BACKEND DEEP DIVE

INTENTSERVICE.JS — INTENT CLASSIFICATION

Uses keyword and regex pattern matching — no AI needed for this step. Deterministic, fast, and unit-testable.

Intent	Triggers
greeting	hi, hello, hey, good morning/evening
gratitude	thank you, thanks, great, awesome
booking_request	book, reserve, I want a room
availability	check availability + dates present
availability_needs_details	availability mentioned but no dates
general_question	everything else

RETRIEVALSERVICE.JS — KNOWLEDGE BASE RETRIEVAL

Implements keyword-overlap scoring: tokenises the guest's question, scores each KB chunk by how many question words appear in it, returns top-N chunks above a threshold. Embeddings were deliberately not used — the KB has ~15 facts, making keyword overlap faster, cheaper, and fully deterministic.

LLMSERVICE.JS — GEMINI INTEGRATION + FALLBACK

```
callLLM(question, contextChunks, conversationHistory)
├─ Builds system prompt: "Answer ONLY from this hotel data: {context}"
├─ Includes last 10 conversation turns as message history
├─ Calls Gemini API (models/gemini-2.0-flash)
├─ On 429/503/network error → falls back to mockAnswer(question, contextChunks)
└─ Returns { text, usedFallback, source }
```

HALLUCINATION PREVENTION

System prompt explicitly restricts the model to provided context. If retrieval returns no matching chunks, the LLM is skipped entirely. Availability numbers never pass through the model.

AVAILABILITYSERVICE.JS — ROOM INVENTORY

Fully deterministic — no AI involved. Validates dates and guest count, computes nights, returns deterministic inventory. Room types: Standard Queen (\$149), Deluxe King (\$189), Triple Room (\$219), Family Suite (\$279).

CONVERSATIONSTORE.JS — MULTI-TURN CONTEXT

Simple in-memory Map: `conversationId → [{ role, content }]`. Keeps last 10 turns per conversation. Resets on server restart — intentional at demo scale.

7. FRONTEND DEEP DIVE

PAGES/INDEX.JS — LAYOUT

Two-column full-viewport layout. Left panel (340px): hotel brand, location, concierge hours, check-in/out times, status badge, and developer credit footer. Right panel: chat header with status pill and the full `ChatWindow` component.

CHATWINDOW.JS — CHAT STATE

Manages all chat state: `messages[]`, `conversationId`, `loading`, `showAvailabilityForm`. Suggestion chips are shown before the first user message; the "Check Room Availability" quick-action button replaces them after. Every request sends `X-API-Key` header.

MESSAGEBUBBLE.JS — MESSAGE RENDERER

Kind	Renders
<code>text</code>	Standard chat bubble (user right, assistant left)
<code>typing</code>	Animated three-dot loading indicator
<code>availability</code>	Results card with per-room pricing and status
<code>error</code>	Red error bubble with "Unable to connect" header

Variable	Value	Used for
<code>--gold</code>	#C59B27	Accents, borders, interactive highlights
<code>--ink</code>	#0B192C	Primary text
<code>--cream</code>	#F7F4EF	Right panel background
<code>--panel</code>	#FFFFFF	Chat bubbles, form cards
<code>--serif</code>	Cinzel	Hotel name, developer credit
<code>--sans</code>	Plus Jakarta Sans	All body text

8. AI INTEGRATION

MODEL USED

Google Gemini 2.0 Flash ([models/gemini-2.0-flash](#)) via the Gemini REST API. Chosen for its free tier (20 req/day), strong instruction-following, and fast response times. The app works fully without any API key via KB fallback mode.

AI VS. DETERMINISTIC SPLIT

Task	AI Used?	Reason
Intent detection	No	Deterministic regex is faster and testable
Knowledge base retrieval	No	Keyword overlap; DB scale doesn't need vectors
Date / guest validation	No	Pure arithmetic
Room availability calculation	No	Deterministic business logic
Natural-language answer generation	Yes	Only task requiring fluent prose
Availability number formatting	No	Structured data inserted directly

SYSTEM PROMPT

You are a professional hotel guest assistant for Harborview Grand Hotel. Answer ONLY using the CONTEXT below. If the context does not contain the answer, say clearly that you don't have that information and suggest contacting the front desk – do not guess or invent details. Never state or imply room availability; availability is handled separately. Do NOT use any emojis. Keep answers concise, formal, and helpful.

CONTEXT:
{retrieved knowledge-base chunks}

FALLBACK CHAIN

```
Guest question
→ Retrieve KB chunks
→ If no chunks match      → return fixed "I don't have that information" (no LLM)
→ If chunks found        → call Gemini API
    → Gemini succeeds      → return LLM response
    → Gemini fails         → return mockAnswer() from KB chunks directly
```

9. KNOWLEDGE BASE

Stored in `backend/src/data/knowledgeBase.json`. Contains four top-level keys:

- **hotel** — name, checkInTime, checkOutTime, cancellationPolicy, breakfast, amenities array
- **rooms** — Standard Queen (\$149, max 2), Deluxe King (\$189, max 2), Triple Room (\$219, max 3), Family Suite (\$279, max 4)
- **faqs** — common questions with canonical answers

```
{
  "hotel": {
    "checkInTime": "3:00 PM",
    "checkOutTime": "11:00 AM",
    "cancellationPolicy": "Free cancellation up to 48 hours before check-in...",
    "breakfast": "Complimentary continental breakfast included..."
  },
  "rooms": [
    { "id": "triple-room", "name": "Triple Room",
      "maxGuests": 3, "pricePerNight": 219 }
  ],
  "faqs": [
    { "question": "Is there parking?", "answer": "..." }
  ]
}
```

10. API REFERENCE

POST /API/CHAT

Request body

```
{
  "message": "What time is check-in?",
  "conversationId": "optional-existing-id",
  "availability": {
    "checkIn": "2027-03-01",
    "checkOut": "2027-03-03",
    "adults": 2
  }
}
```

Response — General answer

```
{
  "conversationId": "uuid",
  "type": "answer",
  "reply": "Check-in is at 3:00 PM.",
  "meta": { "usedFallback": false, "source": "gemini" }
}
```

Response — Availability results

```
{
  "conversationId": "uuid",
  "type": "availability",
  "reply": "Here's what I found for 2 night(s), 2 guest(s):",
  "availability": {
    "checkIn": "2027-03-01", "checkOut": "2027-03-03",
    "adults": 2, "nights": 2,
    "results": [
      { "roomId": "standard-queen", "roomName": "Standard Queen",
        "maxGuests": 2, "pricePerNight": 149, "totalPrice": 298,
        "available": true, "unitsAvailable": 2 }
    ]
  }
}
```

Response — Validation error (400)

```
{ "error": { "code": "INVALID_REQUEST",
  "message": "`message` is required and must be a non-empty string." } }
```

GET /API/HEALTH

```
{ "status": "ok" }
```

11. TEST COVERAGE

BACKEND — JEST (27/27 PASSING)

Suite	Tests	What's covered
<code>chat.test.js</code>	12	Full API integration: FAQ answers, follow-ups, availability, validation, fallbacks, 404, 500
<code>availabilityService.test.js</code>	12	Room results, night calculation, validation edge cases, determinism
<code>retrieval_intent.test.js</code>	3	Intent detection, KB retrieval scoring

FRONTEND — PLAYWRIGHT (4/4 PASSING)

Test	What's covered
Load the chat interface	Page renders, welcome message visible, suggestion chips present
Send a message and receive a response	Full round-trip to backend, response bubble appears
Show availability form	Clicking the availability chip shows the date form
Handle error state gracefully	Error bubble shown when backend is unreachable

MANUAL SCENARIOS (ALL PASS)

Scenario	Result
All 6 required questions answered	PASS
Typing indicator during loading	PASS
Follow-up questions use conversation context	PASS
Invalid dates rejected with clear message	PASS
Out-of-scope question — fallback, no hallucination	PASS
Backend offline — frontend error bubble shown	PASS

12. ASSIGNMENT COMPLETION CHECKLIST

Requirement	Status	Evidence
Chat UI for hotel questions	COMPLETE	pages/index.js, ChatWindow.js
All 6 required sample questions handled	COMPLETE	Suggestion chips + backend intent routing
Room availability form (dates + guests)	COMPLETE	AvailabilityForm.js
Availability results displayed per room	COMPLETE	AvailabilityResults in AvailabilityForm.js
Multi-turn conversation context	COMPLETE	conversationStore.js, conversationId
LLM integration (Gemini)	COMPLETE	llmService.js
Knowledge base (hotel facts)	COMPLETE	knowledgeBase.json
Grounded answers — no hallucination	COMPLETE	System prompt + retrieval gate
Fallback for unsupported questions	COMPLETE	FALLBACK_MESSAGE in llmService.js
Loading / error states in UI	COMPLETE	Typing bubble + error bubble
Clean structured API responses	COMPLETE	POST /api/chat documented contract
Automated backend tests (27)	COMPLETE	npm test → 27/27
Playwright E2E tests (4)	COMPLETE	npm run test:e2e → 4/4
README with setup instructions	COMPLETE	README.md
Architecture documentation	COMPLETE	docs/ARCHITECTURE.md
Test plan with scenarios	COMPLETE	docs/TEST_PLAN.md
Requirement audit	COMPLETE	docs/REQUIREMENT_AUDIT.md
GitHub repository	COMPLETE	github.com/Sumitboii/HARBORVIEW-GRAND
Deployed live	COMPLETE	harborview-grand.onrender.com

INTENTIONALLY OUT OF SCOPE

Item	Reason
Real booking / payment	Brief asks for availability only, not actual booking
Persistent DB for conversations	Demo scale; in-memory is sufficient; noted as a production gap
Embedding-based retrieval	KB has 15 facts; keyword overlap is more transparent and testable
Auth / rate limiting	Optional in brief; time-boxed assignment

13. DESIGN DECISIONS & TRADE-OFFS

TWO-COLUMN LAYOUT

A sidebar with hotel info + chat panel mirrors real hotel concierge apps. It keeps hotel context always visible while the chat stays focused. Each sidebar item is a passive information display; the chat handles all interaction.

KEEPING AVAILABILITY DETERMINISTIC

AI models can hallucinate numbers. A guest making a booking decision based on wrong room availability would be a serious UX failure. The availability check is a pure function: same input always gives same output. The LLM never sees or touches inventory numbers.

KEYWORD RETRIEVAL VS. EMBEDDINGS

The knowledge base has ~15 facts. Vector embeddings would add a vector database, an extra API call per query, and a model dependency for embedding generation — with zero real accuracy gain at this scale. At production scale with hundreds of KB entries, embeddings would become necessary.

IN-MEMORY CONVERSATION STORE

A real DB would require schema migrations, connection management, and deployment setup for a demo-scale project. The trade-off is explicit in the README: conversations reset on server restart, which is acceptable for the assignment scope.

GEMINI INSTEAD OF OTHER LLMS

Gemini 2.0 Flash has a free tier (20 req/day), strong instruction-following, and fast latency. The app works without any API key at all via KB fallback mode, so graders can run the full app with zero credentials.

14. HOW TO RUN LOCALLY

PREREQUISITES

- Node.js 18+
- npm
- A Gemini API key (optional — app works without it)

BACKEND

```
cd backend
npm install
cp .env.example .env          # optional: add GEMINI_API_KEY=your_key
npm run dev                   # http://localhost:4000
```

FRONTEND (SECOND TERMINAL)

```
cd frontend
npm install
cp .env.example .env          # sets NEXT_PUBLIC_BACKEND_URL=http://localhost:4000
npm run dev                   # http://localhost:3000
```

Open **<http://localhost:3000>** and start chatting.

RUN TESTS

```
# Backend (27 tests)
cd backend && npm test

# Frontend E2E (4 tests) – requires both servers running
cd frontend && npm run test:e2e
```

ENVIRONMENT VARIABLES

Variable	Where	Required	Description
<code>GEMINI_API_KEY</code>	backend/.env	No	Gemini API key — app works without it (KB fallback mode)
<code>GEMINI_LLM_MODEL</code>	backend/.env	No	Default: models/gemini-2.0-flash
<code>PORT</code>	backend/.env	No	Default: 4000
<code>ALLOWED_ORIGIN</code>	backend/.env	Production	Frontend URL for CORS allowlist
<code>NEXT_PUBLIC_BACKEND_URL</code>	frontend/.env	Yes	Backend API base URL

Designed and built by **Sumit Kumar Singh**
Harborview Grand — AI Guest Concierge System